



Inspiring Excellence

CSE370 : Database Systems

Project Report

Project Title : BRACU Club Management System

Group No : 01, CSE370 Lab Section : 11, Spring 2026		
ID	Name	Contribution
23201418	Anindya Debnath Ani	Club features, and OCA Module
24101461	Raaida Rubaba Sharif	Admin and Announcement Panel, and Club components
24101231	Sheikh Musa Rahman	Landing pages, and Club attributes

Table of Contents

Section No	Content	Page No
1	Introduction	3
2	Project Features	4
3	ER/EER Diagram	5
4	Schema Diagram	6
5	Normalization	7 - 8
6	Frontend Development	9 - 12
7	Backend Development	13 - 14
8	Source Code Repository	15
9	Conclusion	15
10	References	16

Introduction

Between coordinating events, handling budgets, tracking expenses, and keeping everyone in the loop, managing student clubs can be a very hectic and demanding task. The administrative burden can quickly become overwhelming, especially when it runs on paper forms and email threads.

The BRACU Club Management System (BRACU CMS) was built to make this entire process easier. It is a full-stack web application that brings all club-related workflows into one centralized platform, connecting Club Presidents and panel members directly with the Office of Co-curricular Activities (OCA) at BRAC University.

Rather than waiting days for a form to get signed, club representatives can log in, submit event proposals, request budgets, and check approval statuses in real time. On the other side, OCA staff can review requests, broadcast announcements, and keep track of all registered clubs from a single dashboard.

The system is built around two primary user roles:

Club Panel: Members who represent their club can propose events, manage their roster, record expenses, add sponsors, and file post-event reports — all without stepping into an office.

OCA Admin: University staff who oversee all clubs. They approve or reject event proposals and budget requests, post university-wide announcements, and get a bird's-eye view of club activity across campus.

At its core, this project is about reducing friction. The less time student leaders spend navigating bureaucracy, the more they can invest in actually running meaningful events and building their communities.

Project Features

ID, Name	Features [3 per member]	
23201418, Anindya Debnath Ani	Ft 1	Login & Authentication: Users sign in with their credentials and are routed to the correct dashboard based on their role — OCA Admin or Club Panel — using a secure, cookie-based session system.
	Ft 2	Finance Dashboard: Club presidents can see a visual summary of their allocated budget, total spending, and remaining funds through interactive charts and a detailed budget table.
	Ft 3	Post-Event Reports: After an event wraps up, clubs can submit a written report along with supporting files (images or PDFs). OCA can then mark the report as accepted or declined.
24101461, Raaida Rubaba Sharif	Ft 1	Announcements by Admins: Admins can post announcements and updates which instantly show up on every club president's dashboard.
	Ft 2	Club Directory: Showcases all the registered clubs, their information such as their president, the amount of registered members, and their financial rundown.
	Ft 3	Event Calendar: A monthly calendar to showcase pending and upcoming club events. It also helps club administrators to make sure no scheduling conflicts take place.
24101231, Sheikh Musa Rahman	Ft 1	Event Proposals: Club members can fill out and submit an event request form with details like venue, date, and a description. The request then goes to OCA for review.
	Ft 2	Sponsor Tracking: Clubs can log their external sponsors — including contact info and what the sponsorship covers — and link each deal to a specific event.
	Ft 3	Budget Requests: Once an event is approved, clubs can submit a budget request for it. OCA reviews the request and allocates funds through the same portal.

ER/EER Diagram

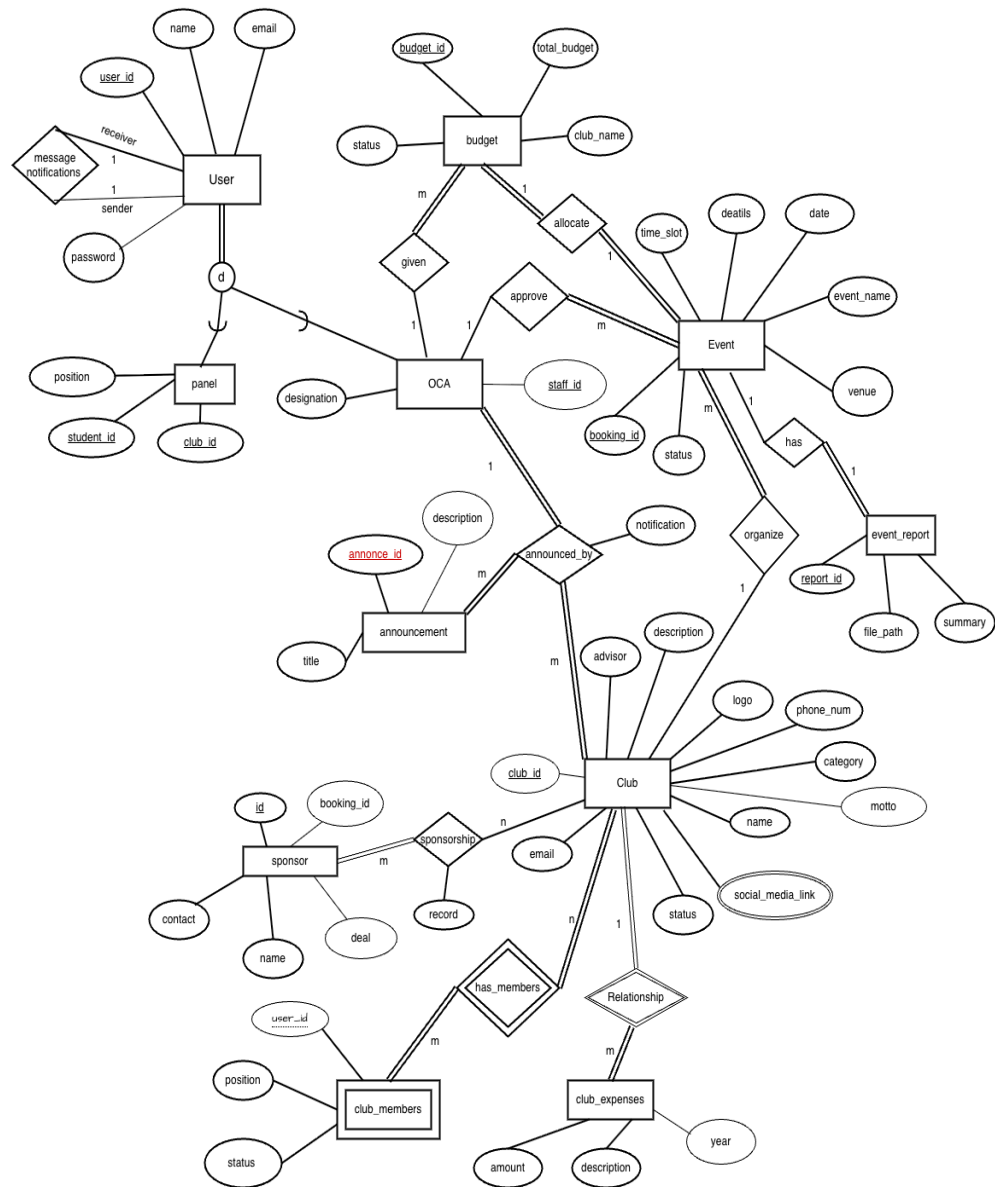


Figure : EER Diagram

Schema Diagram

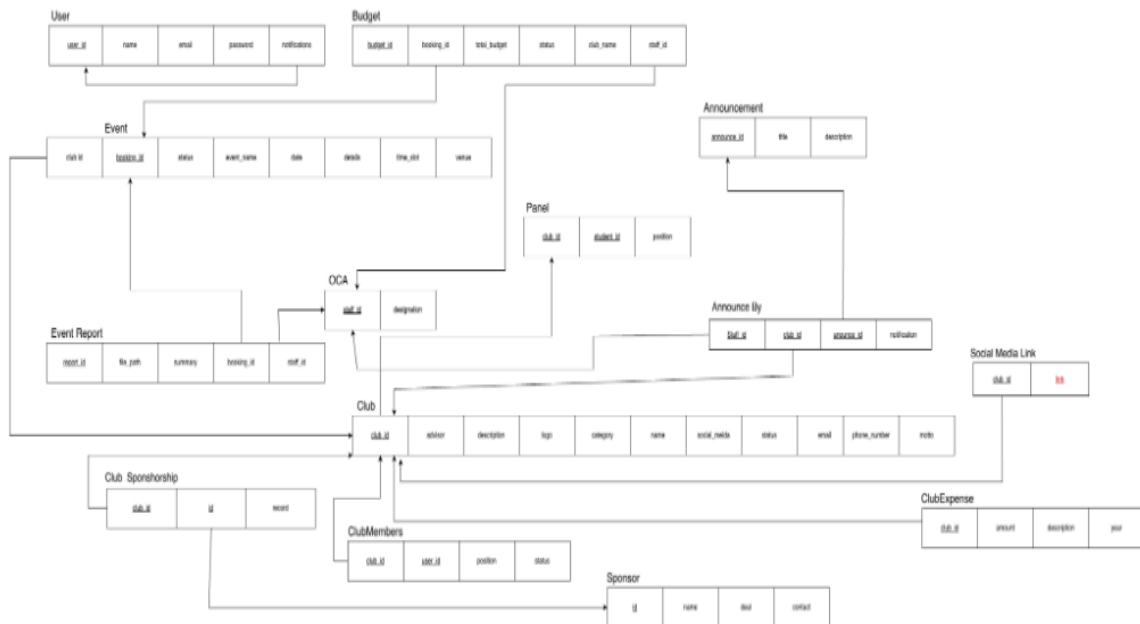


Figure : Schema

Normalization

- a. Explain if your converted Schema is in 1NF or not. If not, decompose it to 1NF.
 - b. Explain if your converted Schema is in 2NF or not. If not, decompose it to 2NF. Can there be any partial functional dependencies in your relational schema?
 - c. Explain if your converted Schema is in 3NF or not. If not, decompose it to 3NF. Can there be any transitive dependencies in your relational schema?
-
- a) **First Normal Form** : The schema is in 1NF. Every column holds a single, indivisible value — there are no lists or comma-separated entries hiding inside a field. The notifications column in Users is stored as JSON, but the database treats it as one atomic value, so it does not break 1NF from a relational standpoint.
 - b) **Second Normal Form** : The schema is in 2NF. Every table uses a single-column surrogate primary key (an auto-incrementing integer), which means partial dependency is not even possible — you cannot have a non-key attribute that depends on only part of a composite key when there is no composite key to begin with. All non-key columns depend entirely on their table's primary key.
 - c) **Third Normal Form** : The schema is in 3NF. There are no transitive dependencies — no situation where column A determines column B, which in turn determines column C, with B not being a key. One thing worth flagging: the Budget table stores a club_name column. This is intentionally denormalized for display purposes, but the authoritative source remains the Club table. It does not create a true transitive dependency since club_name is not used to derive any other attribute within Budget.

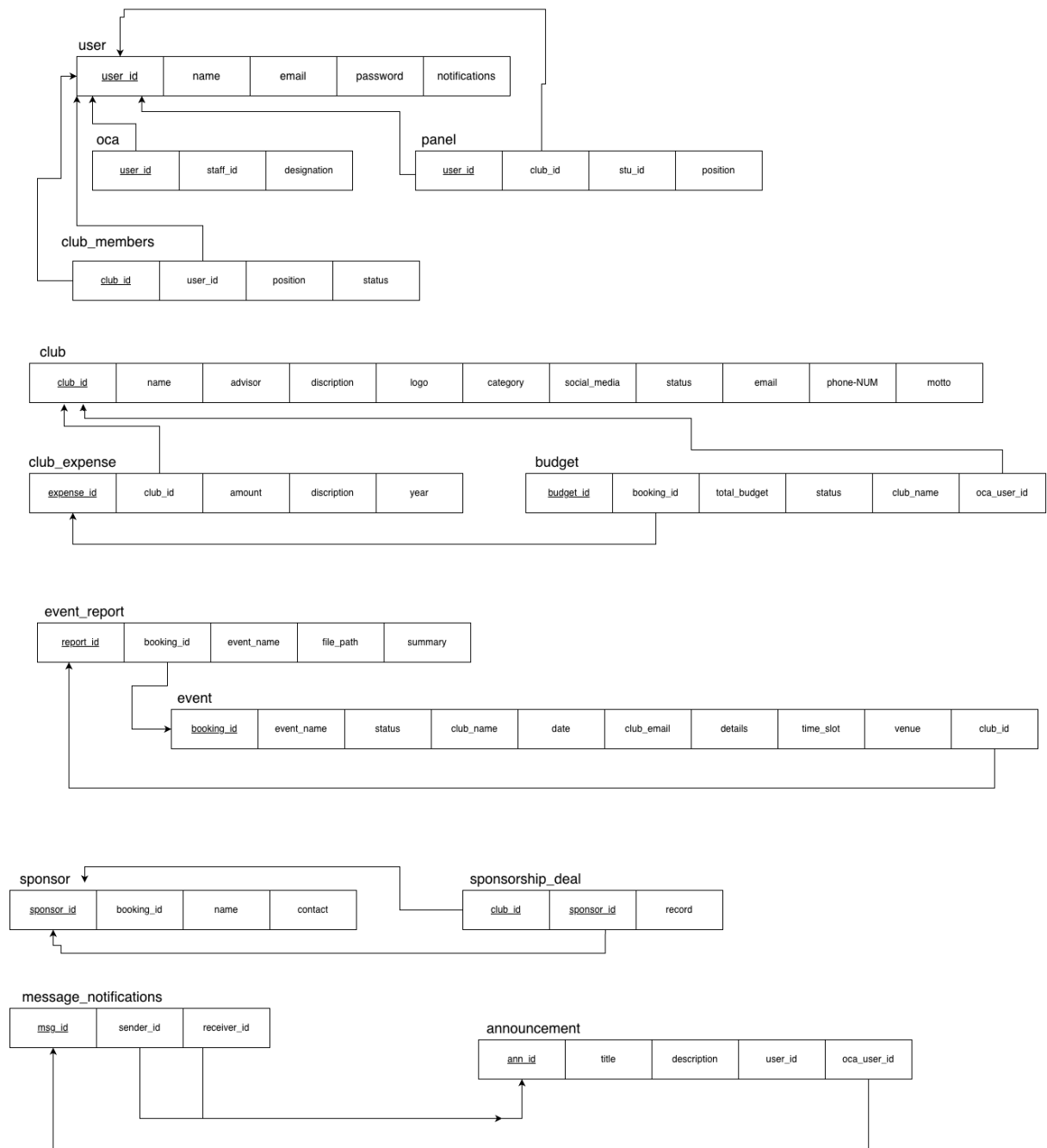


Figure : Normalization

Frontend Development

Contribution of ID : 23201418, Name : Anindya Debnath Ani

Initialised the Next.js repository and set up the core structure, and was responsible for the following parts:

Home Page: The public-facing landing page. Includes a full-screen hero section, a feature overview, and call-to-action buttons.

Login Page: A clean sign-in screen that authenticates the user and sends them to the right dashboard depending on their role.

OCA Dashboard: The admin's main overview - shows all registered clubs, pending requests, and quick-access stats.

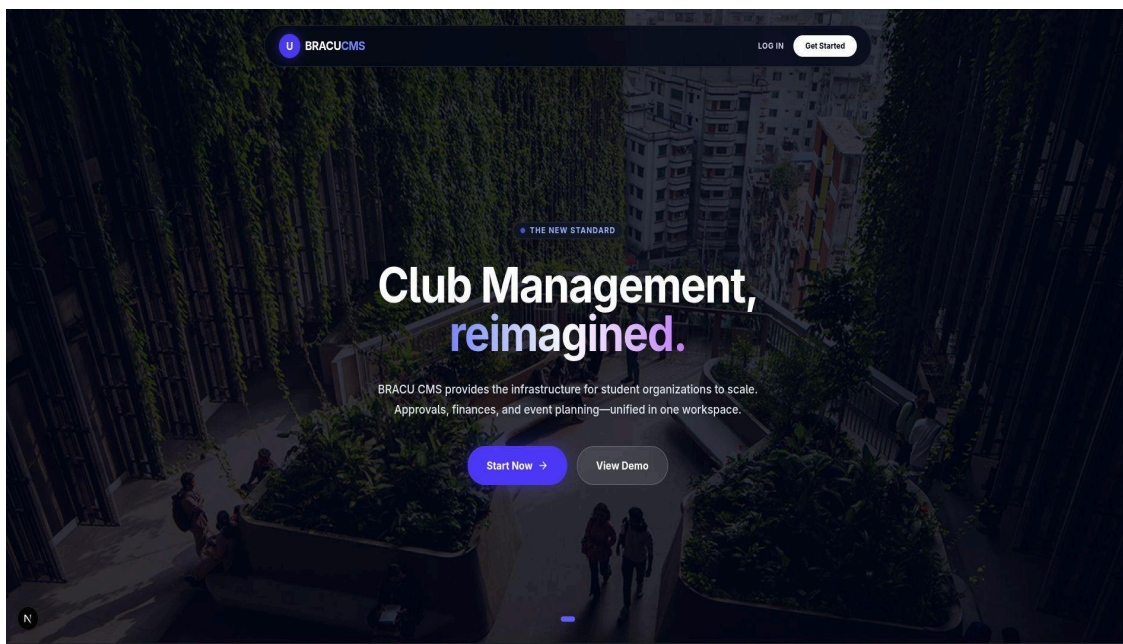
Register Club Page: A form that lets OCA staff add new clubs to the system.

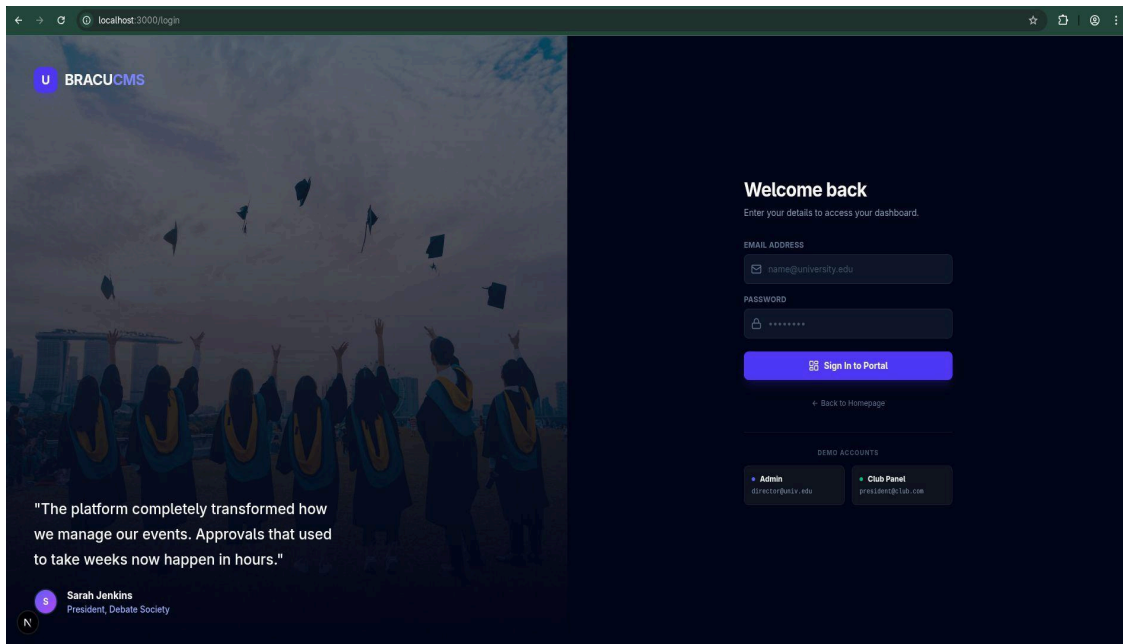
Finance Page: Displays a club's budget breakdown with a bar chart for yearly spending and a table of all budget requests.

Reports Page: Lets club members submit post-event reports and attach evidence files.

Club Home Page: The first thing a club president sees after logging in - shows announcements and a summary of active events.

Shared Components: BudgetTable, ExpenseChart, and the Sidebar navigation component, all reused across multiple pages.





Contribution of ID : 24101461, Name : Raaida Rubaba Sharif

Constructed the main club management features such as CreateEvent, Budget, CalendarView and more :

Budget Page: A form where clubs can submit a budget amount for a specific approved event, with an option to cancel mid-way.

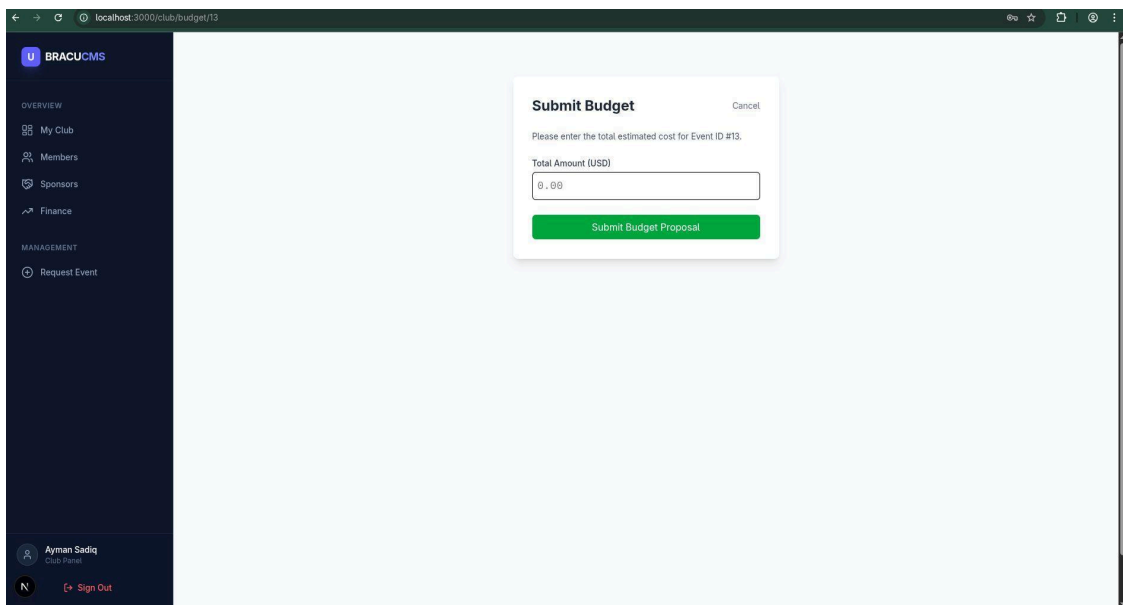
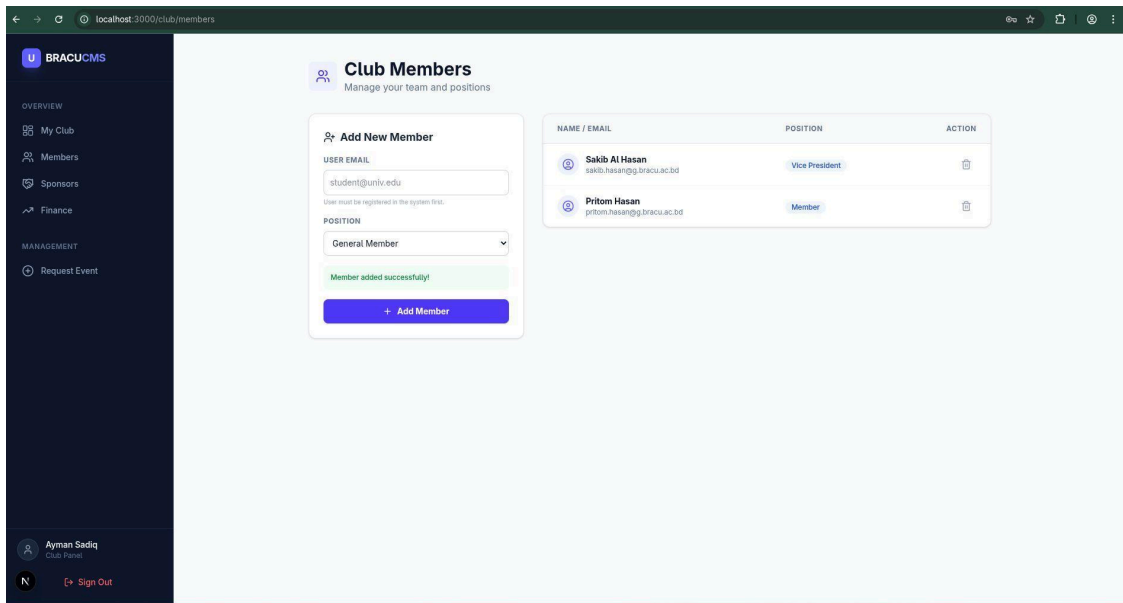
Create-Event Page: The event proposal form — collects the event name, venue, date, time slot, and a description before sending it to OCA.

Members Page: Shows the club's current roster and lets authorized users add new members by entering their university email.

CalendarView Component: A navigable month view that highlights days with scheduled events.

AddExpenseInline Component: A compact expense-entry form that sits right on the Finance page without navigating away.

AddMemberForm Component: A small form embedded on the Members page to add users who are already registered in the system.



Contribution of ID : 24101231, Name : Sheikh Musa Rahman

Focused on the sponsorship and communication side of the app, building:

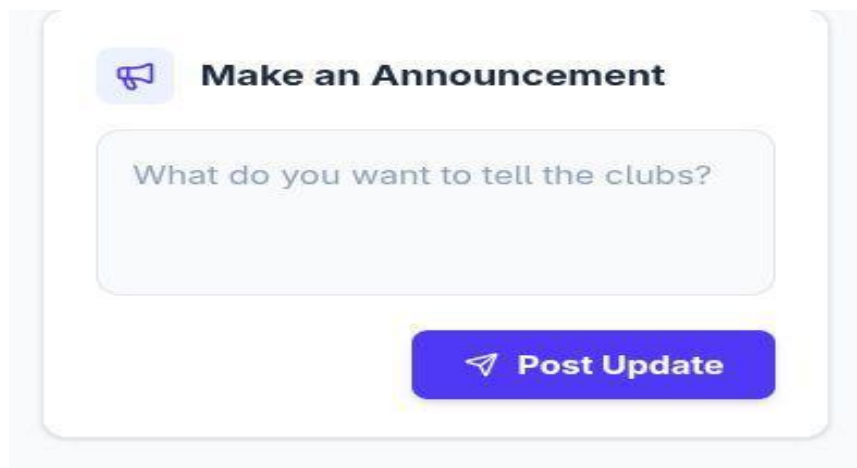
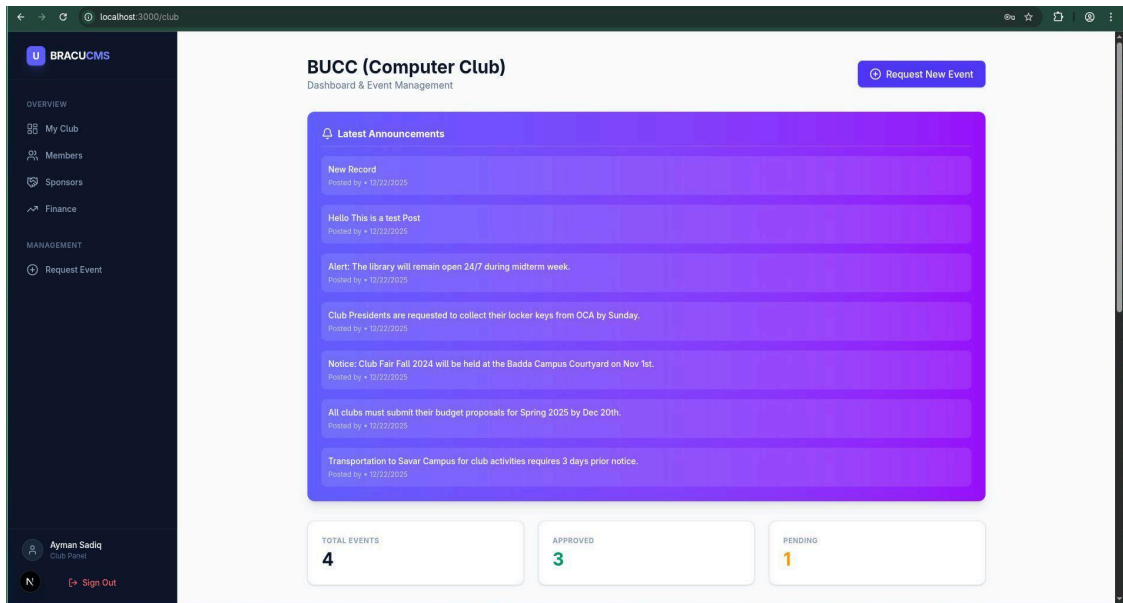
Sponsors Page: Shows all sponsors linked to the club's events in a clean table, with an option to add a new sponsor from the same view.

Add Sponsor Flow: A form that captures the sponsor name, contact email, the event it belongs to, and the nature of the deal.

MemberList Component: A presentational component that renders the club roster in a consistent format across pages.

AnnouncementForm Component: The input UI for OCA admins to write and post announcements to all clubs.

AnnouncementFeed Component: Displays incoming OCA announcements on the Club Home page in reverse chronological order.



Backend Development

Contribution of ID : 23201418, Name : Anindya Debnath Ani

Database Setup: Created the MySQL connection pool using mysql2 with a connection limit of 10. Added environment-aware initialization so dev mode does not spawn duplicate connections on hot reload.

Authentication Logic: Wrote the login flow that checks both the OCA and Panel tables to figure out who the user is and what role they should have. Sessions are persisted using HTTP-Only, Secure, SameSite cookies - safer than localStorage and immune to XSS.

File Upload Pipeline: Built the server-side handler for event report attachments using Node.js fs/promises. It reads the incoming file as a Buffer, creates the target directory if it does not exist, and writes the file to disk safely.

Member & Sponsor Management: Wrote the server actions for adding and removing club members, and for linking sponsors to specific event booking IDs.

Aggregated Queries: Authored multi-table JOIN queries (like getClubEventBudgets) that pull together event, budget, and club data in a single database call for the finance dashboard.

Event & Budget Lifecycle: Handled the full flow for events and budgets: creation, fetching pending records for OCA review, and status updates (Approved / Rejected).

Contribution of ID : 24101461, Name : Raaida Rubaba Sharif

Event Server Actions: Implemented submitEventAction (submitting a proposal) and reviewEventAction (OCA approves or rejects). Wrote the query that returns all approved events for the public calendar.

Announcement System: Implemented postAnnouncementAction with a role check - if the session does not belong to an OCA admin, the action throws an error before touching the database

Membership Backend: Wrote the logic for fetching a club's member list, removing a member, and the 'Add Member' action — which checks that the user being added actually exists in the system before creating the record.

Expense Analytics: Created the addExpense action and a getExpenseStats aggregation query that groups spending by year so Recharts can render it as a bar chart.

Sponsorship & Reports: Connected sponsor records to event bookings and wrote the query that joins sponsor data with event report data for the post-event view.

Contribution of ID : 24101231, Name : Sheikh Musa Rahman

Sponsor & Budget Queries: Wrote the getSponsors and getClubEventBudgets queries - both use multi-table JOINS to link sponsor or budget records back to the club through its event history.

Expense Recording: Built the addExpenseAction server action, which automatically calculates the fiscal year from the current date and validates the session before writing anything to the database.

Budget Workflow: Added server-side handling for budget submissions and approvals, including a guard that prevents a club from submitting two budget requests for the same event.

Budget Status Reporting: Wrote the aggregation query that compares requested vs. allocated amounts per event, which powers the status badges on the Finance page.

Cache Invalidation: Integrated Next.js revalidatePath calls inside the relevant server actions so pages like Finance and Club Home refresh automatically after a change - no manual reload needed.

Source Code Repository

https://github.com/smrwazi/CSE370_Project

Conclusion

Student clubs play a big role in campus life, but the administrative side of running one can be very slow and paper-heavy. This project aims to change that for BRAC University by building a practical, database-driven web application that handles the full club management lifecycle.

This application covers a lot of ground - secure role-based login, event proposal and approval flows, budget tracking with visual analytics, sponsor management, post-event reporting, real-time OCA announcements, and a live club directory. They are all backed by a normalized relational database and server-side logic that keeps data consistent and the interface responsive.

Some key points :

- **Faster turnaround:** Approval decisions that used to take days now take minutes, since everything is visible to OCA in real time.
- **Cleaner finances:** Every expense and budget allocation is logged and visible, making it much harder for funds to fall through the cracks.
- **Better information flow:** The announcement system means clubs are always up to date with university notices without relying on forwarded emails.
- **Institutional continuity:** Past event reports and financial records stay in the system, giving future club leaders a useful historical reference.

References

1. Next.js. (2024). Server Actions and Mutations. Vercel Documentation.
<https://nextjs.org/docs/app/building-your-application/data-fetching/server-actions-and-mutations>
2. Oracle Corporation. (2024). MySQL 8.0 Reference Manual.
<https://dev.mysql.com/doc/refman/8.0/en/>
3. Meta Open Source. (2024). React Documentation — Server Components and Hooks.
<https://react.dev>
4. Tailwind Labs. (2024). Tailwind CSS Documentation. <https://tailwindcss.com/docs>
5. sidorares. (2024). node-mysql2 — MySQL client for Node.js. GitHub.
<https://github.com/sidorares/node-mysql2>
6. Recharts. (2024). Recharts Documentation — Composable charting library for React.
<https://recharts.org>